SOFTWARE ENGINEERING LAB REPORT

Component Diagram

LAB 8 | Case Study: Food Delivery Application

Project Overview

The Food Delivery Application allows customers to discover restaurants, browse menus, place and pay for orders, track delivery, and provide feedback. Restaurants manage menus and incoming orders, while delivery partners accept assignments and update delivery status. An administrator supervises users, restaurants, payments, and operational issues.

Submitted by

Team Member	Roll Number
Deepak Singh	202401100200123
Deepak Maurya	202401100200121
Deepak Nimesh	202401100100122
Deependra Ahirwar	202401100200124

Department of Computer Science & Engineering

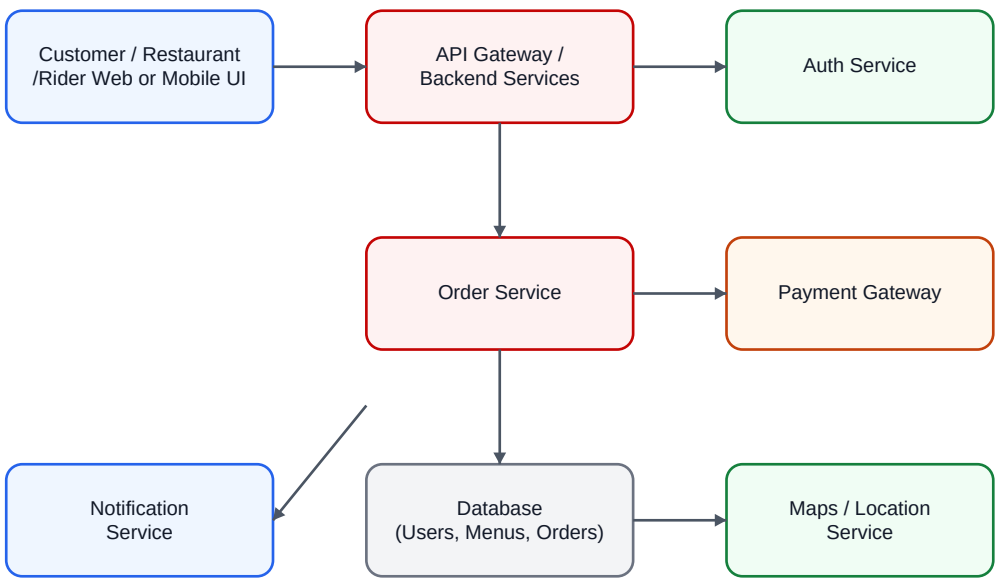
1. Aim

To illustrate the major software components of the Food Delivery Application and their dependencies.

2. Components and Responsibilities

Component	Responsibility
Customer / Restaurant / Rider UI	Screens for browsing, order management, menu management, and delivery updates.
API Gateway / Backend	Authentication, request validation, rate limiting, routing, and shared middleware.
Authentication Service	Login, registration, roles, access tokens, and session handling.
Order Service	Cart validation, order creation, status transitions, and order history.
Payment Adapter	Integrates with external payment gateway; handles callbacks and idempotency.
Notification Service	Sends order confirmations, status alerts, and delivery notifications.
Database	Persists users, restaurants, menu items, orders, payments, and audit events.
Maps / Location Adapter	Geocoding, route estimation, and rider location features.

3. Component Diagram



Components communicate through APIs; external services are accessed via adapters.

4. Design Considerations

• Use API contracts between UI and backend so components can evolve independently.

• Keep payment-provider logic behind an adapter to simplify provider changes and testing.

• Protect endpoints with role-based access control and validate all client input.

• Use durable order status events and retries for notifications; avoid duplicate orders/payments through idempotency.

• For larger deployments, scale stateless API services horizontally and monitor latency, error rate, and queue backlog.

5. Result

The component diagram gives a high-level view of the application architecture and the key integrations needed for a reliable food delivery workflow.